

Optimization Dynamics on Representational Collapse

Pei-Chi Huang
peichi@umich.edu

1 Introduction

Deep neural networks (DNNs) are traditionally characterized as hierarchical feature extractors, where each successive layer is thought to learn increasingly complex and task-specific data representations. To facilitate the training of these complex hierarchies, a wide array of optimization techniques has been proposed. However, **the selection of an appropriate optimizer is a critical architectural decision**, as varying optimization dynamics can fundamentally alter the model’s trajectory and internal representation building. Suboptimal choices can lead to unstable convergence or premature overfitting during the early stages of training.

Our work extends prior empirical studies by **evaluating the tunnel effect across a wider range of optimization methods and training configurations**. We examine whether this effect is a universal property of overparameterized networks or whether optimizer-specific dynamics, such as adaptive simplicity bias or Muon’s update behavior, can suppress or alter its emergence. Through model-optimizer comparisons, we assess whether the “extractor-tunnel” split is invariant or algorithm-dependent.

The main contributions of this paper are summarized as follows:

1. **Cross-Optimizer Validation of the Tunnel Effect:** We conduct a comparative analysis of tunnel development across standard and adaptive optimization algorithms, including an evaluation of the most recent Muon optimizer.
2. **Granular Epoch-Wise Analysis:** We provide a more detailed examination of low-rank characteristics in representation and weight dynamics to contrast how different optimizers influence tunnel formation. Specifically, we describe temporal changes across training phases.

All the related scripts are available at: <https://github.com/Peggy1210/tunnel-effect-exp>

2 The Tunnel Effect

In overparameterized deep neural networks, increasing depth does not necessarily correlate with an indefinite increase in representational complexity. Instead, these models exhibit a tendency to simplify representations as depth increases, focusing primarily on discriminative patterns required for supervised tasks. This phenomenon, characterized by a functional bifurcation of the model’s architecture, is known as **the tunnel effect** [1].

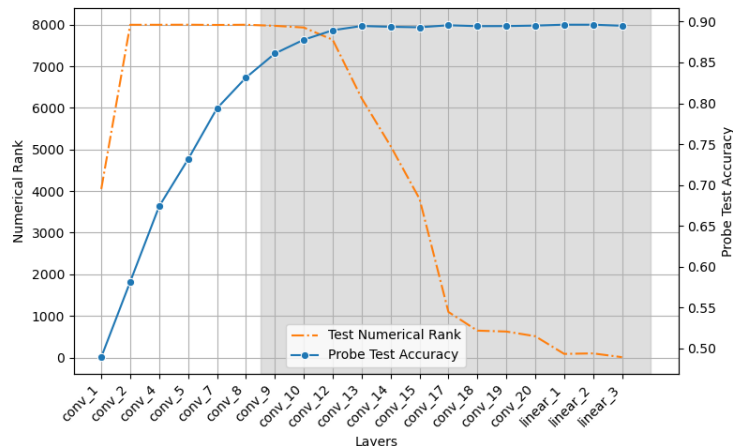


Figure 1: The tunnel effect for VGG19 trained with SGD and momentum. In the tunnel (shaded area), the performance of linear probe (blue line) attached to each layer saturates, and the representation rank (red dashed line) is steeply reduced.

Sufficiently deep networks naturally split into two distinct components with diverging behaviors:

- **The Extractor:** The initial layers of the network are responsible for constructing linearly-separable representations. During this phase, performance (as measured by linear probing accuracy) increases rapidly as the model builds high-level features.
- **The Tunnel:** Subsequent layers serve to propagate representations toward the output while significantly compressing them. These layers contribute minimally to the final task performance but play a critical role in the model’s structural dynamics.

The tunnel is defined by several unique characteristics that differentiate it from the extractor:

- **Representation Compression and Rank Collapse:** Within the tunnel, the numerical rank of representations is steeply reduced, often collapsing to a value approximately equal to the number of classes in the dataset.
- **Early Emergence:** The split between the extractor and the tunnel is not a late-stage development; it emerges early in the training process and persists throughout optimization.
- **Stabilization of Weights:** Parameter changes within the tunnel layers are significantly smaller than those in the extractor, indicating that these deeper layers stabilize their configuration early in training.

The original characterization of the tunnel effect was primarily established using **Stochastic Gradient Descent (SGD) with or without momentum**. It remains unclear whether this effect is an inherent structural property of deep networks or a specific artifact of SGD-based optimization dynamics. We hypothesize that the choice of optimizer is a critical factor that directly influences training dynamics and representational structure.

Consequently, this paper conducts a systematic comparison of various optimization techniques, ranging from **standard SGD** to **adaptive methods** and the novel **Muon optimizer**, to determine if certain algorithms can suppress or shift the emergence of the tunnel.

3 Experiment

This section describes the experimental framework for replicating the "Tunnel Effect" and the additional optimizers evaluated, with parameters tuned for each model.

3.1 Tasks

Our primary investigation uses the **CIFAR-10 dataset**, a standard benchmark for image classification. The dataset consists of **60,000 RGB images across 10 classes**, with 50,000 images for training and 10,000 for testing. Each image has a resolution of **32×32 pixels with 3 channels**.

Although datasets such as CIFAR-100 and CINIC-10 have been documented as viable for tunnel analysis, this study reports results only on CIFAR-10 due to computational resource constraints. However, this does not limit the generality of our experiment, as the prevalence of the tunnel effect has been shown across different datasets. Code is still available to extract these additional datasets.

The data were preprocessed using **standardization with the dataset means and standard deviations**. To isolate the effects of optimization dynamics, **no data augmentation** was applied. We used the full image size and all 3 RGB channels without converting images to grayscale.

3.2 Model Architectures

We evaluate the tunnel effect across three architectural families to determine if representational collapse is an invariant property of deep learning or specific to certain architectural biases. Detailed architectural specifications, including layer counts, channel dimensions, and normalization protocols, are provided in Appendix A.1.

- **Multilayer Perceptrons (MLP):** Representing a baseline for unstructured, fully connected networks [2]. Due to the lack of spatial inductive bias, we hypothesize that MLPs will exhibit the most pronounced and rapid tunnel formation.
- **VGG-style CNNs:** Representing deep, sequential convolutional hierarchies without skip connections [3, 4]. We expect these models to demonstrate a "mid-hierarchy" transition, where early layers act as local feature extractors and deeper layers serve as task-specific compressors.
- **Residual Networks (ResNet):** Representing modern architectures with identity mappings [5]. We hypothesize that shortcut connections will mitigate the intensity of the tunnel effect by allowing representational information to bypass deeper layers, potentially maintaining a higher numerical rank throughout the network compared to plain-network counterparts.

3.3 Optimizations Strategies

All models were trained using Cross-Entropy loss with a label smoothing factor of 0.01. To investigate the sensitivity of the tunnel effect to various optimization landscapes, we evaluated a spectrum of algorithms ranging from standard gradient descent to modern orthogonalization methods. The detailed parameters used for training can be found in Appendix A.2.

We categorize our selection into three groups:

- **Non-stochastic Baseline: Full-batch Gradient Descent (GD)** is used to isolate the effect of stochasticity.
- **First-order Stochastic Methods: SGD** and **SGD with Momentum** represent the standard baselines upon which the original tunnel effect was established.
- **Adaptive and Second-order-like Methods: Adam** and **AdamW** are included to analyze the impact of adaptive learning rates and decoupled weight decay. Finally, we incorporate **Muon**, a recently proposed optimizer that utilizes Newton-style updates via orthogonalization, to test if preserving representational "health" during updates can mitigate the early emergence of the tunnel.

We hypothesize that optimizers with higher simplicity bias (like standard SGD) may accelerate the formation of the tunnel. Conversely, methods that enforce diverse feature updates, particularly Muon, are expected to maintain higher numerical rank in the deeper layers for a longer duration, potentially delaying the onset of representational collapse.

3.4 Measurements and Metrics

To analyze the internal dynamics of each model, we extract the hidden representations h_l from every layer l for a given input batch. For **MLP-12**, this includes all 12 hidden layers plus the final classifier (13 layers total). For **VGG-19**, we extract from the 16 convolutional layers and the three subsequent fully connected layers (19 layers total). For **ResNet-34**, representations are extracted from the output of each residual block and the final classification layer.

3.4.1 Linear Probing and the Tunnel Entry Point

To measure the task-specific information contained within each layer, we employ **linear probing** [6]. This technique involves freezing the weights of the pre-trained model and training a single linear classifier on top of the representations produced by layer l . The accuracy of this probe serves as a proxy for the linear separability and quality of the features at that specific depth. Details for training are given in A.2.5.

Following *Masarczyk et al. (2023)*, we define the tunnel entry point as the first layer whose linear probing accuracy reaches 95% of the network's final output accuracy. Layers beyond this threshold are considered part of the tunnel.

3.4.2 Supporting Metrics for Representational Analysis

To provide a more granular view of the tunneling behavior, we utilize additional metrics that track the geometric and dimensionality changes in the representation space.

- **Numerical Rank:** We estimate the numerical rank of representation matrices to measure degeneracy. The rank is calculated as the number of singular values exceeding a threshold of $\sigma_1 \times 10^{-3}$, where σ_1 is the leading singular value. To maintain efficiency, we randomly sample 8,000 features per layer for this calculation.
- **Inter- and Intra-class Variance:** We compute the dispersion between different classes (inter-class) and the homogeneity within a single class (intra-class). This determines the degree to which representations have collapsed into task-specific clusters.
- **Centered Kernel Alignment (CKA) Similarity:** CKA [7] similarity is used to quantify the representational similarity between different layers. This metric allows us to identify blocks of similar representations, particularly within the tunnel.
- **Singular Value Spectrum:** We compute the ordered singular values $\sigma_1 \geq \sigma_2 \geq \dots$ for each layer’s representations. The decay rate of this spectrum characterizes the distribution of information across the feature space. A relatively flat spectrum signifies that the representation’s variance is distributed across many dimensions (high isotropy), whereas a rapidly decaying spectrum indicates that the representation is concentrated in a few dominant directions.
- **Top-K Energy:** Similar to the singular values, we measure the top-k spectral energy $\sum_{i=1}^k \sigma_i^2 / \sum_i \sigma_i^2$ of layer representations. This corresponds to the fraction of total variance explained by the top k principal directions. A higher top-k energy indicates that most of the representation’s variance is captured by a small number of directions, i.e., stronger low-rank structure.

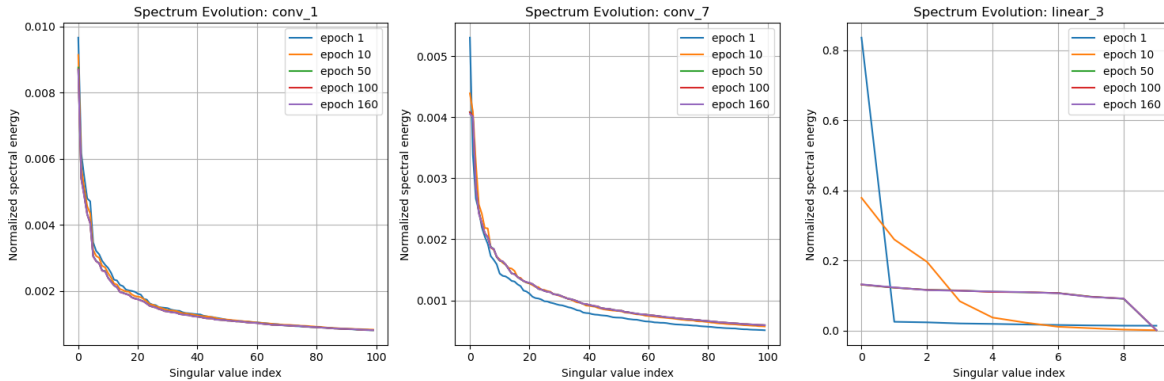


Figure 2: Singular value spectrum of the representation output from the first-layer extractor (left), the tunnel start layer (middle), and the deep tunnel layer (right), trained on VGG-19 with SGD.

4 Results

4.1 Training Results

All models converged across the tested optimizers but showed clear overfitting (see Appendix B), reaching near-perfect training accuracy while underperforming on test data. This suggests that the networks transitioned from learning generalizable features to memorizing the training distribution. Although this reduced test performance, the resulting high-capacity memorization state is useful for our later catastrophic forgetting and continual learning analyses.

The empirical convergence rates of the evaluated optimizers align closely with their respective theoretical profiles (Figure 3). **Full-batch GD exhibits the slowest convergence velocity**, albeit with the highest trajectory smoothness due to the absence of stochastic mini-batch noise. In contrast, **the Muon optimizer serves as the most rapid optimization method**, followed closely by the adaptive methods, Adamw and Adam. While vanilla **SGD demonstrates slower initial convergence**, it reaches the lowest terminal training loss at the earliest epoch among all tested methods. SGD with momentum displays significant initial velocity but tends to plateau at a relatively high loss value; however, this is subsequently mitigated by the learning rate scheduler. **The scheduled step-decay effectively facilitates further minimization**, allowing the model to escape the plateau and achieve a competitive final loss.

Across all models, **the MLP-12 showed different training dynamics**. While training loss steadily decreased, test loss followed a U-shaped curve before rising sharply above its initial value. Despite this divergence, test accuracy plateaued around 0.40 - 0.50. This behavior reflects the MLP’s **lack of spatial inductive bias**. Unlike convolutional models, which capture local image structure, the MLP treats pixels as independent features. As a result, it enters a tunneling regime with shallow, low-rank representations that are insufficient for robust image classification.

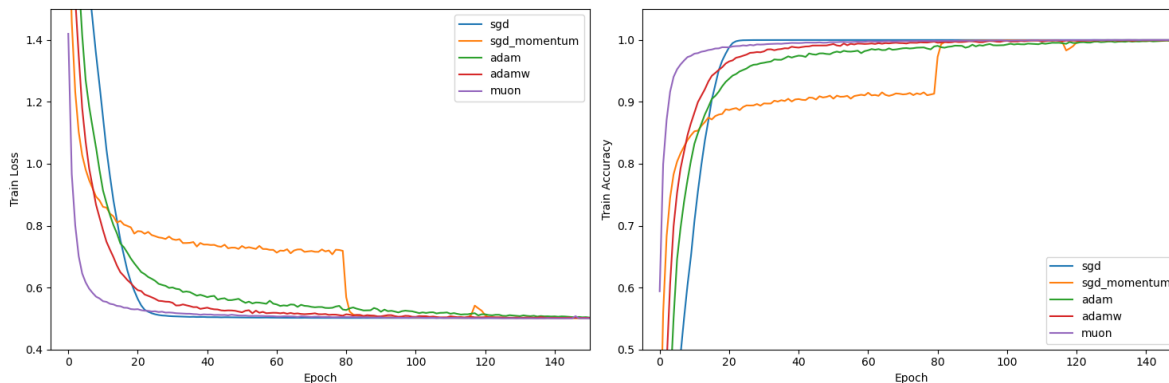


Figure 3: Training performance curves for VGG-19 with different optimizer settings. **Left:** Training loss. **Right:** Training accuracy.

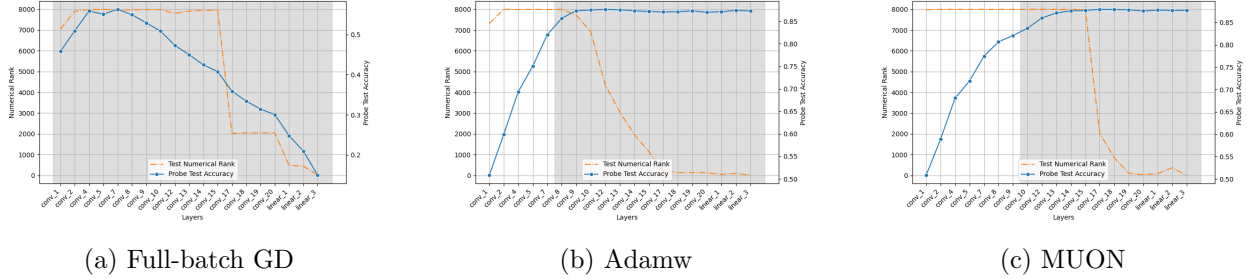


Figure 4: The tunnel effect observed across different optimizers trained on a **VGG-19 architecture**. The shaded area represents the tunnel, the blue lines indicate probing accuracy, and the orange dashed line denotes the numerical rank.

4.2 The Tunnel Effect

Overall, **our results suggest a tunnel effect across optimizers** (Table 1). We observe that linear probing accuracy increases in deeper layers. For deeper layers, the accuracy remains mostly unchanged, changes only slightly, or sometimes even decreases.

For the numerical rank, we use a test set of 10,000 samples and randomly select 8,000 samples for feature extraction. A high rank close to 8,000 suggests that the image representation is **nearly full rank**. Since this is close to the number of samples, it suggests that the model refrains from learning compact representations and instead tends to memorize data characteristics. This also indicates that our model is highly overparameterized. **At later stages, the rank collapses**, the tunnel suppresses the rank to fit the number of classes (Figure 1, Figure 4).

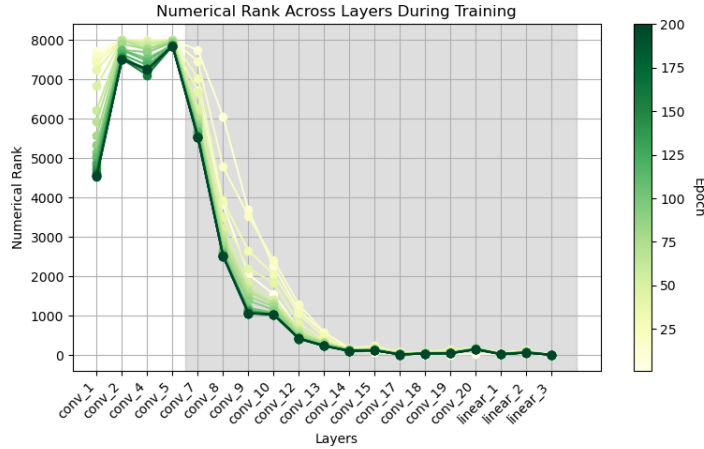
We observe a mismatch between the 95% linear probing tunnel entry point and structural rank collapse, especially in Full-batch GD and MLP-12. Also, the tunnel start in ResNet-34 is especially late. This indicates that the tunnel is a multi-stage process: features first become linearly separable, then later undergo compression. Since linear probing measures separability while numerical rank measures dimensionality, high accuracy can appear before redundant directions are removed. Finer identification of tunnel start may be needed. For example, numerical rank and isotropy may better capture the structural tunnel than probing accuracy alone.

Interestingly, the **inter-class variance remains low across almost all layers**, with a slight increase in later stages. In contrast, the **intra-class variance decreases and then remains constant in later stages** (Appendix C). This supports the original claim that the extractor tends to reduce intra-class variance significantly, while the tunnel further refines classification by maintaining low intra-class variance and slightly increasing inter-class variance.

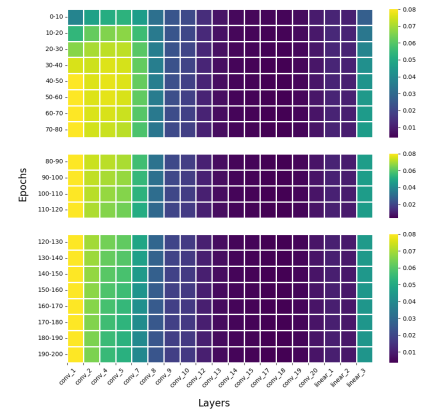
The original paper suggests that **the tunnel should maintain similar rank throughout training**. Instead, we observe that the tunnel rank changes substantially (Figure 5a). This may be partly because our extractor is nearly full rank, making rank changes in the extractor difficult to observe directly. We also computed the norm of weight differences (Figure 5b). **The extractor changes noticeably, while tunnel layers undergo only a small shift**. This further suggests that the tunnel forms early in training and remains relatively stable across epochs.

Architecture	# layers	Optimizer	>0.95	>0.98
MLP-12	13	Fullbatch	1 (0.0%)	1 (0.0%)
		SGD	3 (15.4%)	5 (30.8%)
		SGD + Momentum	2 (7.7%)	3 (15.4%)
		Adam	2 (7.7%)	3 (15.4%)
		Adamw	4 (23.1%)	10 (69.2%)
		MUON	1 (0.0%)	2 (7.7%)
VGG-19	19	Fullbatch	1 (0.0%)	1 (0.0%)
		SGD	5 (21.1%)	5 (21.1%)
		SGD + Momentum	7 (31.6%)	9 (42.1%)
		Adam	5 (21.1%)	6 (26.3%)
		Adamw	6 (26.3%)	6 (26.3%)
		MUON	8 (36.8%)	9 (42.1%)
ResNet-34	34	Fullbatch	3 (8.8%)	3 (8.8%)
		SGD	21 (61.8%)	25 (73.5%)
		SGD + Momentum	29 (85.3%)	31 (91.2%)
		Adam	19 (55.9%)	29 (85.3%)
		Adamw	21 (61.8%)	29 (85.3%)
		MUON	23 (67.6%)	29 (85.3%)

Table 1: The tunnel effect across models with different optimizers.



(a) Numerical Rank Evolution



(b) Weight Difference Norm

Figure 5: Temporal evolution of the tunnel effect in VGG-19 using Adamw. **Left:** Numerical rank across 200 epochs (light to dark curves indicate temporal progression); the shaded region denotes the tunnel. **Right:** Heatmap of weight differences ($\|\Delta W\|$) showing early parameter stabilization in tunnel layers (right) compared to the active extractor layers (left)

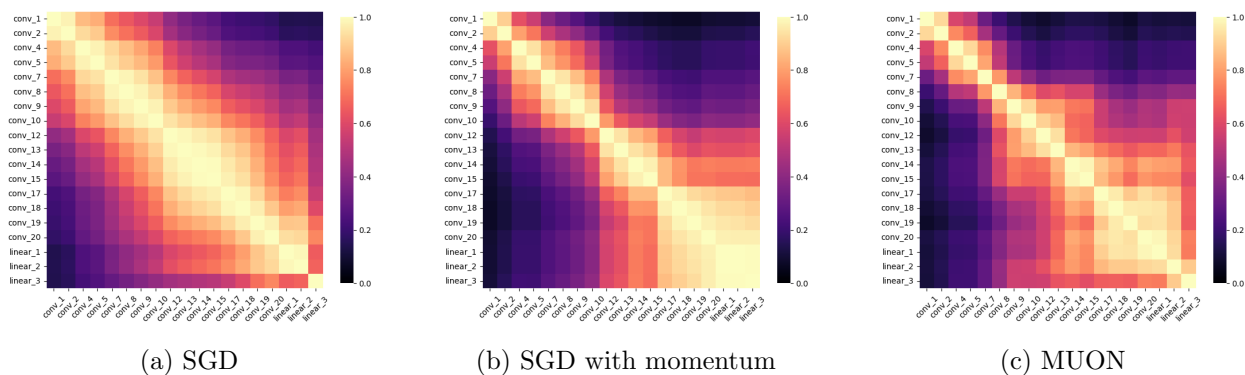


Figure 6: Representational similarity across VGG-19 layers. Brighter means more similar.

4.2.1 Supporting Experiments

In the initial extractor layers, low CKA values between non-adjacent layers show that the model is actively building hierarchical features and reshaping the data manifold. As the network enters the tunnel, a strong diagonal block of high CKA similarity, often above 0.9, appears (Figure 6). This suggests that once the features become sufficiently linearly separable, deeper layers largely “idle,” passing nearly unchanged representations to the classifier.

To further analyze this effect, we added metrics such as the singular value spectrum (Figure 2) and top-K energy (Figure 7). In the extractor phase, the spectrum remains relatively flat, indicating that information is spread across many feature directions. In the tunnel, however, the spectrum decays rapidly, concentrating variance into a few dominant directions. Notably, this low-rank structure appears early in training, with decay visible by epoch 10, suggesting that the network quickly finds a task-specific subspace and later mainly refines compression within it.

The top-K energy again confirms this trend. In tunnel layers, the first 10 singular values capture nearly 95% - 100% of the total energy, showing that task-irrelevant dimensions are largely discarded. By contrast, extractor layers preserve high-dimensional representations, where even the top 100 singular values may explain less than 50%. Overall, these results indicate that the extractor maintains diverse feature information, while the tunnel performs rank compression.

4.2.2 Comparison of Optimizers

Full-batch GD (Figure 4a) behaves differently from the standard tunnel effect. Across architectures, layers reach 95% probing accuracy early, but numerical rank stays high until a sharp drop in the final layers. Without mini-batch noise, full-batch GD appears less prone to early rank collapse and continues using depth for representation learning. It appears to exploit a richer parameter trajectory, allowing the network to continue using depth for feature representation rather than rapidly entering the tunnel.

Vanilla SGD shows the expected tunnel pattern, with early low-rank collapse. Adding momentum delays tunnel entry (Figure 1). Momentum smooths updates, allowing intermediate layers to keep refining features before entering the compressed tunnel regime.

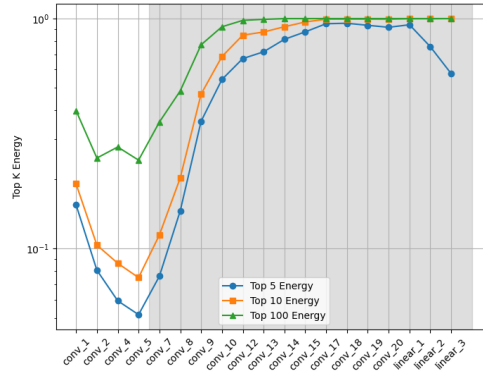


Figure 7: Spectral energy concentration across VGG-19 layers trained with Adam.

Adam and Adamw (Figure 4b) produced similar tunnel profiles in convolutional architectures such as VGG and ResNet, but their behavior differed more clearly in the MLP-12 setting. They show a similar tunnel onset to SGD. However, they achieve more stable final performance, with **AdamW exhibiting a slightly faster rank collapse**, possibly due to decoupled weight decay removing redundant directions.

The Muon optimizer produced the strongest change in tunnel dynamics. In VGG-19 and ResNet-34, Muon consistently shifted tunnel entry deeper, while **maintaining a final-layer numerical rank substantially above the number of CIFAR-10 classes**. This supports Muon’s orthogonalized updates: by discouraging weight updates from concentrating along a few dominant directions, Muon acts as a structural regulator against representational collapse.

4.3 Conclusion & Future Work

Our study confirms that the tunnel effect is a robust phenomenon across various architectural families, yet its temporal and structural evolution is significantly influenced by the choice of optimizer. We demonstrate that **optimization strategies capable of suppressing implicit simplicity bias**, most notably Muon through orthogonalization and SGD with Momentum through inertia, **effectively delay the onset of the tunnel**. By preserving representational diversity deeper into the network hierarchy, these methods allow for a more efficient utilization of overparameterized model capacity compared to standard adaptive or stochastic baselines.

Future research directions include: 1) Our results indicate a misalignment between linear probing accuracy and numerical rank collapse. Future work should explore a **multi-metric definition of the tunnel start** that incorporates spectral decay or isotropy to more precisely capture the transition from feature extraction to compression. 2) **Catastrophic Forgetting**: Examining the link between tunnel length and a model’s susceptibility to forgetting, testing the hypothesis that more compressed tunnels offer less representational buffer for continual learning tasks. 3) **Out-of-Distribution (OOD) Analysis**: Investigating whether the representational health preserved by optimizers like Muon translates to improved robustness on corrupted datasets (e.g., CIFAR-10-C) or better performance under distribution shifts.

References

- [1] W. Masarczyk, M. Ostaszewski, E. Imani, R. Pascanu, P. Miłoś, and T. Trzciński, “The tunnel effect: Building data representations in deep neural networks,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.19753>
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [3] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [4] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International conference on machine learning*. PMLR, 2015, pp. 448–456.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [6] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” *arXiv preprint arXiv:1610.01644*, 2016.
- [7] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, “Similarity of neural network representations revisited,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 3519–3529.
- [8] H. E. Robbins, “A stochastic approximation method,” *Annals of Mathematical Statistics*, vol. 22, pp. 400–407, 1951. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16945044>
- [9] B. Polyak, “Some methods of speeding up the convergence of iteration methods,” *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0041555364901375>
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [11] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [12] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, “Muon: An optimizer for hidden layers in neural networks,” 2024. [Online]. Available: <https://kellerjordan.github.io/posts/muon/>

A Experiment Setup

A.1 Model Architectures

A.1.1 Multilayer Perceptron (MLP-12)

A multilayer perceptron (MLP) [2] is a feedforward neural network architecture that primarily consists of multiple fully connected layers. In our experiments, we analyze an MLP with 12 hidden layers, each with 1,024 neurons. We append a final fully connected layer that serves as the classifier for the 10 output classes.

A.1.2 VGG-19

VGG [3] is a convolutional neural network architecture composed of stacked convolutional layers and pooling layers. We mainly use VGG-19 for our analysis. The architecture consists of five stages of convolutional layers with ReLU activation and max-pooling layers. The five stages use 64, 128, 256, 512, and 512 channels, respectively, with max pooling applied after each stage.

Although this paper focuses on overparameterized models and overfitting is not our main concern, we retain batch normalization after each convolutional layer and before the ReLU activation to improve training stability [4]. The convolutional stages are followed by three fully connected layers. The first two fully connected layers are followed by ReLU activation, while the final layer serves as the classifier.

A.1.3 ResNet-34

ResNet [5] is a convolutional neural network architecture characterized by residual blocks and shortcut connections. We mainly test ResNet-34 in our experiments. ResNet-n is a five-stage network characterized by its depth, with a total of n layers. The initial stage consists of a single convolutional layer with a 7×7 kernel, 64 channels, and ReLU activation, followed by 2×2 max pooling, which reduces the spatial dimensions. The subsequent stages are composed of residual blocks. Each residual block typically contains two convolutional layers and introduces a shortcut connection that skips one or more layers. Each convolutional layer in the residual block is followed by batch normalization and ReLU activation. The remaining four stages consist of 3×3 convolutions with 64, 128, 256, and 512 channels, respectively.

A.2 Optimization Parameters

This section provides a technical overview of the optimization algorithms utilized in our experiments, alongside the specific hyperparameter configurations.

A.2.1 Full-batch Gradient Descent (GD)

Full-batch GD computes the gradient of the loss function with respect to the entire training dataset. While computationally expensive, it provides a "noise-free" baseline to determine if the tunnel effect is a property of the loss landscape itself or a byproduct of stochastic noise during training.

Parameters	MLP	VGG	ResNet
# Epochs	300	300	300
Learning Rate	5e-3	1e-2	1e-2
Momentum	0	0	0
Weight Decay	0	0	0
Scheduler	-	-	-

Table 2: Full-batch GD parameters

A.2.2 Stochastic Gradient Descent (SGD) & Momentum

SGD [8] approximates the gradient using mini-batches, introducing stochasticity that often aids in escaping local minima. SGD with Momentum [9] incorporates a moving average of past gradients to dampen oscillations and accelerate convergence along relevant directions. This has traditionally been the default setting for observing the tunnel effect.

Parameters	MLP	VGG	ResNet
# Epochs	200	160	160
Batch Size	128	128	128
Learning Rate	0.05	0.01	0.01
Momentum	0	0	0
Weight Decay	0	5e-4	5e-4
Scheduler	-	Multistep	Multistep
LR Milestones	-	80, 120	80, 120
LR decay gamma	-	0.1	0.1

Table 3: SGD parameters

Parameters	MLP	VGG	ResNet
# Epochs	200	160	160
Batch Size	128	128	128
Learning Rate	0.05	0.1	0.1
Momentum	0.9	0.9	0.9
Weight Decay	1e-4	5e-4	5e-4
Scheduler	-	Multistep	Multistep
LR Milestones	-	80, 120	80, 120
LR decay gamma	-	0.1	0.1

Table 4: SGD with momentum parameters

A.2.3 Adam & Adamw

Adam (Adaptive Moment Estimation) [10] computes individual adaptive learning rates for different parameters by estimating the first and second moments of the gradients. AdamW [11] improves upon this by decoupling weight decay from the gradient update, which prevents the regularization term from being distorted by the adaptive learning rate scales. These are tested to see if adaptive scaling affects the "compression" behavior in the tunnel.

Parameters	MLP	VGG	ResNet
# Epochs	200	200	200
Batch Size	128	128	128
Learning Rate	1e-3	1e-3	1e-3
Momentum	0	0	0
Weight Decay	1e-4	1e-4	1e-4
Scheduler	Cosine	Cosine	Cosine

Table 5: Adam parameters

Parameters	MLP	VGG	ResNet
# Epochs	200	200	200
Batch Size	128	128	128
Learning Rate	1e-3	1e-3	1e-3
Momentum	0	0	0
Weight Decay	1e-3	5e-4	1e-2
Scheduler	Cosine	Cosine	Cosine

Table 6: Adamw parameters

A.2.4 MUON

Muon [12] is a recent optimizer designed for deep neural networks that applies orthogonalization to the weight updates. By treating the optimization as a form of Newton-style update on the manifold of orthogonal matrices, Muon aims to keep the layers' weights "healthier" and more expressive. In the context of this study, we utilize Muon to investigate whether its ability to preserve update dimensionality can prevent or postpone the rank collapse characteristic of the tunnel effect.

A.2.5 Linear Probing

We attach a linear classifier to each layer to measure the extent to which its representations are linearly separable. Probes are trained for 30 epochs using the Adam optimizer. To follow standard practice in continual learning, we used a small number of epochs, a moderate learning rate, and no weight decay. We sampled 10,000 data points for training due to computational constraints.

Parameters	MLP	VGG	ResNet
# Epochs	200	200	200
Batch Size	128	128	128
Muon LR	5e-4	5e-3	5e-3
Momentum	0.9	0.95	0.95
Nesterov steps	5	5	5
Adamw LR	1e-4	3e-4	3e-4
Adamw WD	1e-3	1e-4	1e-4
Scheduler	Cosine	Cosine	Cosine

Table 7: Muon parameters

Parameters	Value
# Epochs	30
Batch size	512
Learning Rate	1e-3
Weight Decay	0

Table 8: Linear probing training parameters

A.3 Testing Environment

Our experiments were primarily conducted on the Great Lakes cluster. To ensure consistent resource availability and throughput, each training session was limited to an 8 - hour GPU allocation and a maximum of 64 GB of CPU RAM. Under these constraints, training a standard model instance required approximately 2 hour. Post-training representational analysis - including hidden layer extraction, linear probing, and rank estimation - required no less than 3 hours per model.

During the experiments, we also experienced Great Lakes cluster downtimes. We explored alternatives such as Google Colaboratory, which provided access to an NVIDIA T4 GPU, approximately 12 GB of system RAM, and 15 GB of GPU memory. However, this environment was suboptimal because runtime and storage space were limited. Only a few plots, such as the CKA heatmap, were generated with assistance from this environment; each heatmap required up to 20 minutes to plot.

Running the full set of experiments was estimated to require approximately 5 hours for training and analysis per model-optimizer configuration. Across 18 model-optimizer configurations, this amounts to approximately 90 hours. As of the time of this report, more than 1,000 job requests have been submitted, including failed runs.

B Loss and Accuracy Curves

B.1 MLP-12

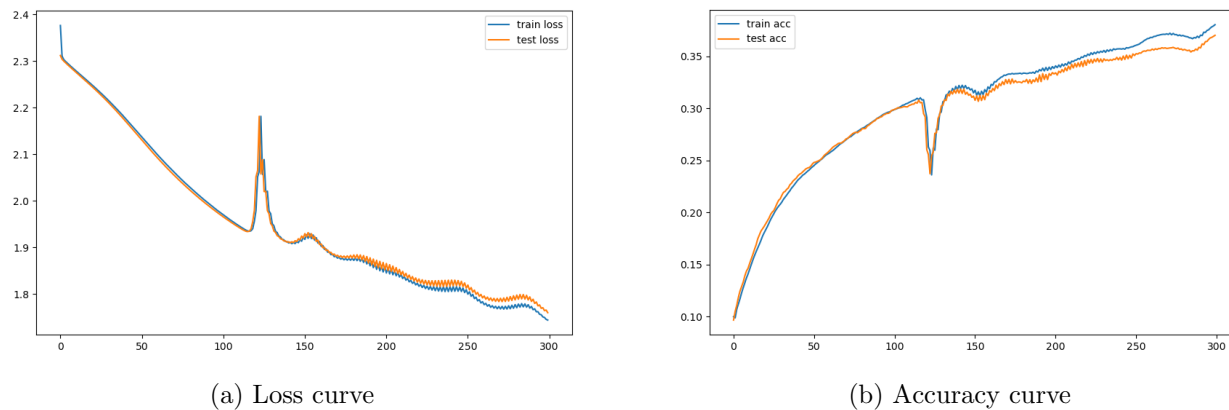


Figure 8: Training and test performance curves for MLP-12 using full-batch GD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

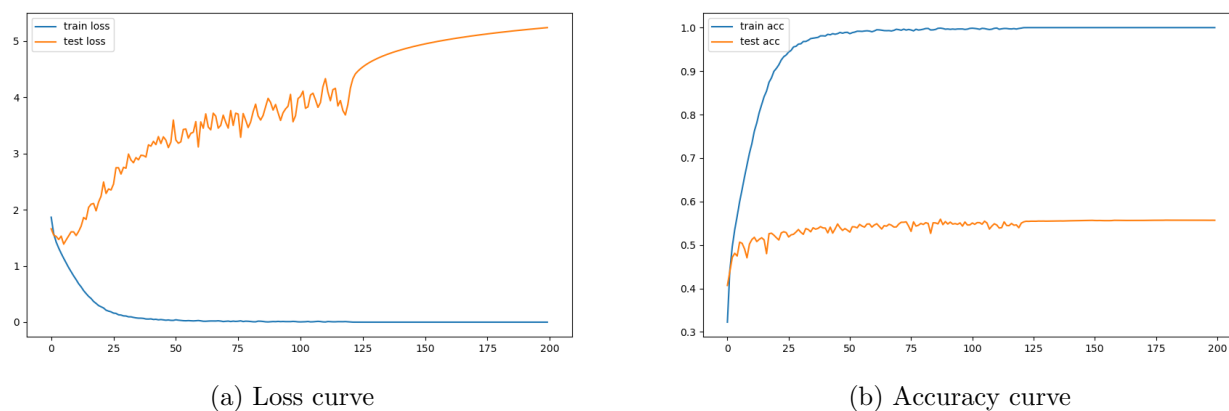
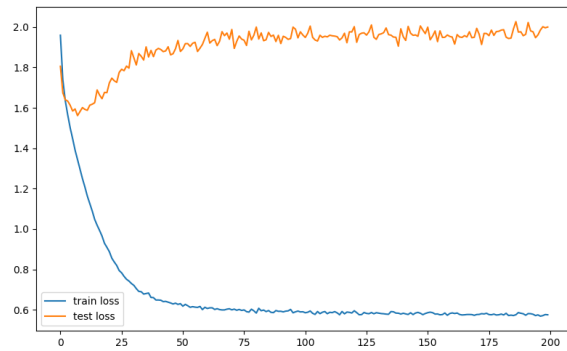
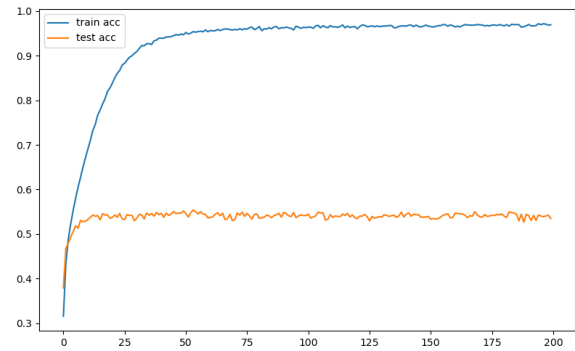


Figure 9: Training and test performance curves for MLP-12 using SGD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

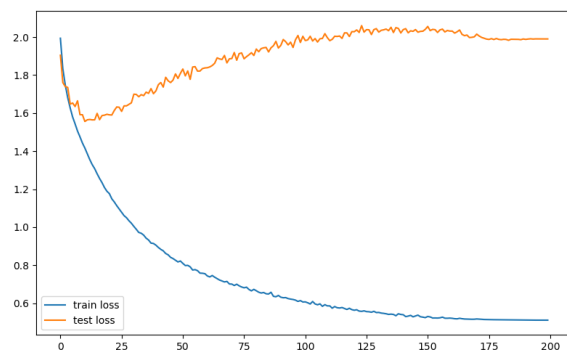


(a) Loss curve

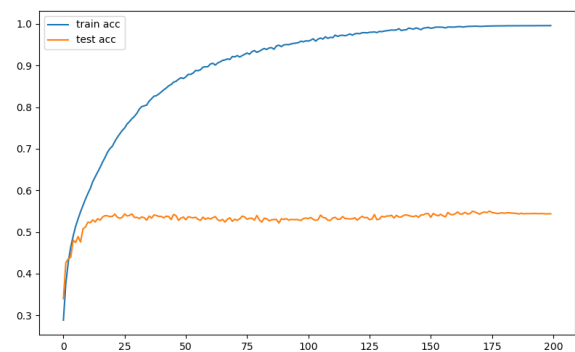


(b) Accuracy curve

Figure 10: Training and test performance curves for MLP-12 using SGD with momentum. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

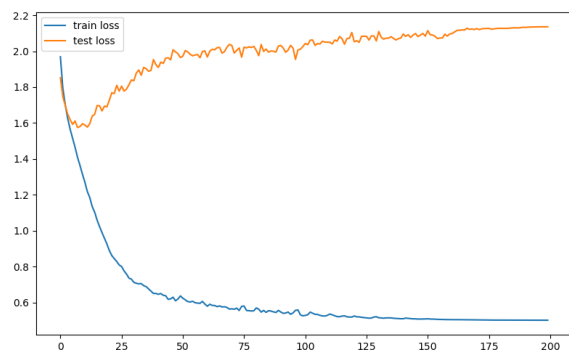


(a) Loss curve

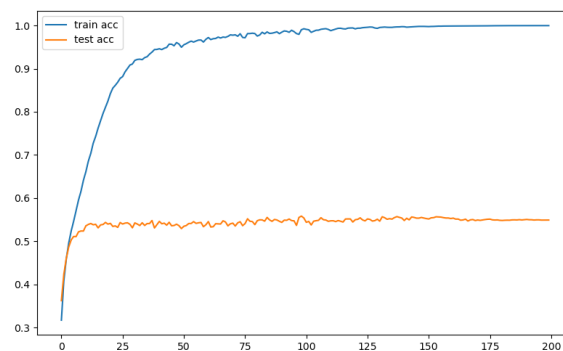


(b) Accuracy curve

Figure 11: Training and test performance curves for MLP-12 using Adam. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

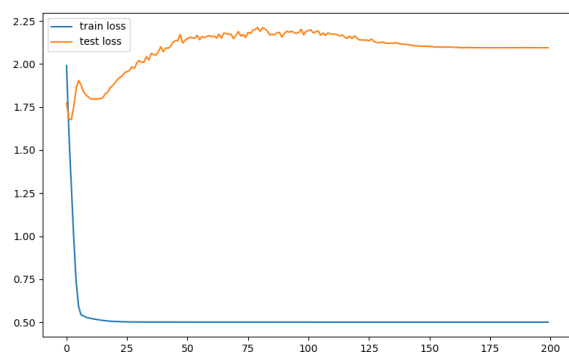


(a) Loss curve

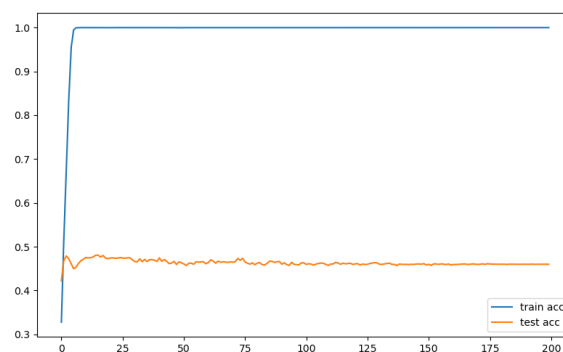


(b) Accuracy curve

Figure 12: Training and test performance curves for MLP-12 using Adamw. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.



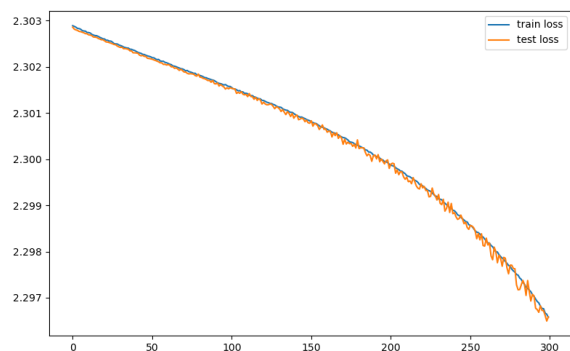
(a) Loss curve



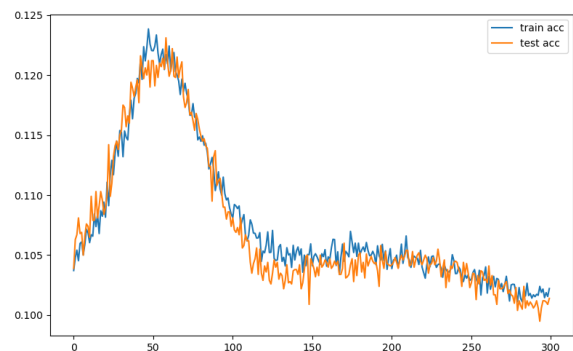
(b) Accuracy curve

Figure 13: Training and test performance curves for MLP-12 using MUON. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

B.2 VGG-19

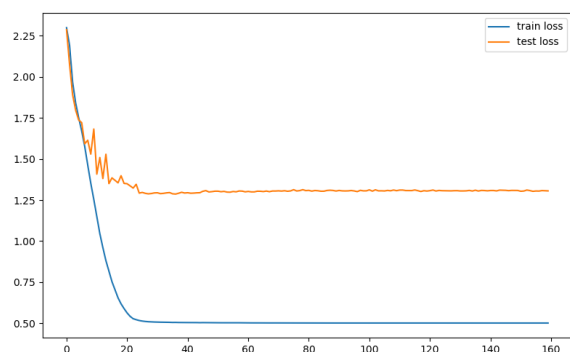


(a) Loss curve

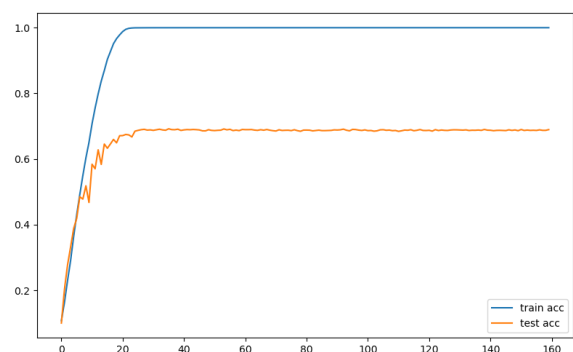


(b) Accuracy curve

Figure 14: Training and test performance curves for VGG-19 using full-batch GD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

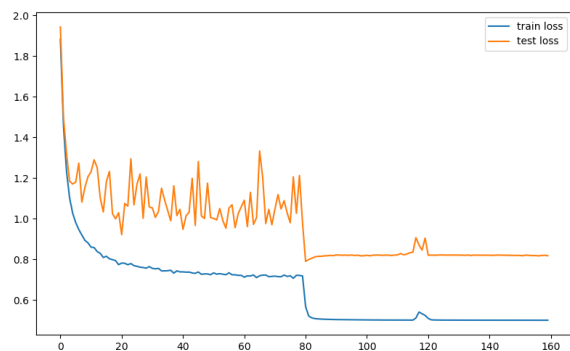


(a) Loss curve

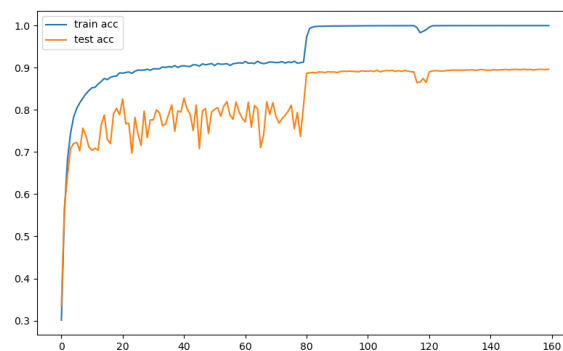


(b) Accuracy curve

Figure 15: Training and test performance curves for VGG-19 using SGD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

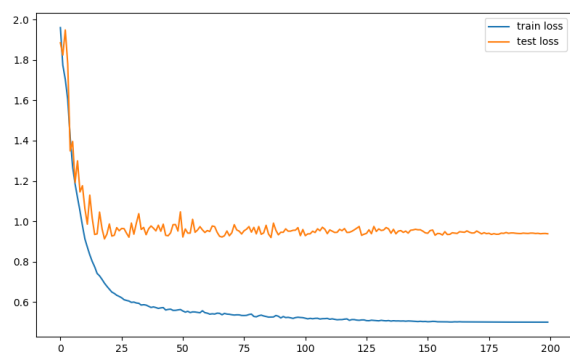


(a) Loss curve

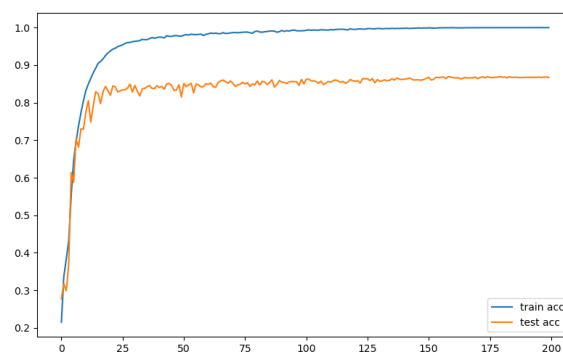


(b) Accuracy curve

Figure 16: Training and test performance curves for VGG-19 using SGD with momentum. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

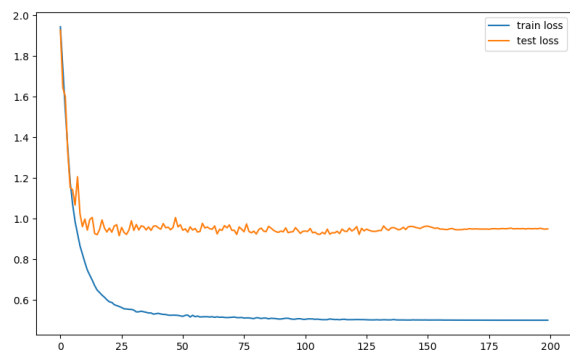


(a) Loss curve

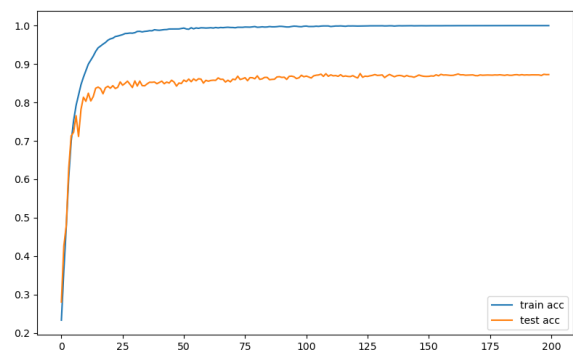


(b) Accuracy curve

Figure 17: Training and test performance curves for VGG-19 using Adam. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

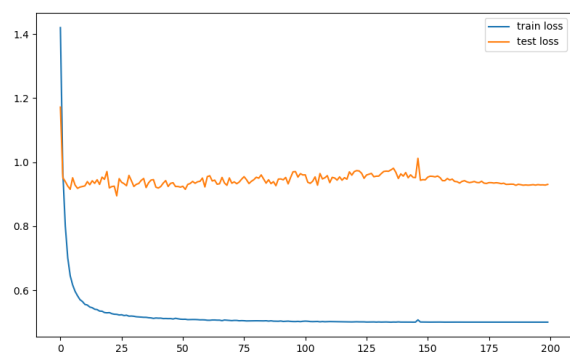


(a) Loss curve

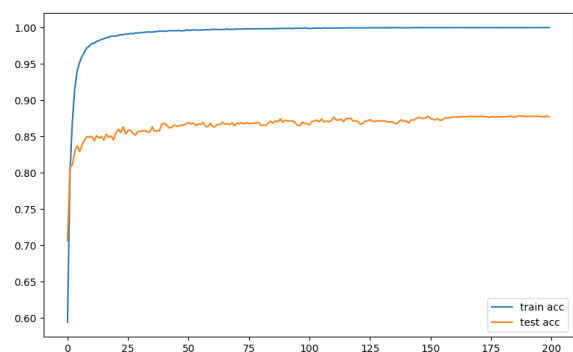


(b) Accuracy curve

Figure 18: Training and test performance curves for VGG-19 using Adamw. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.



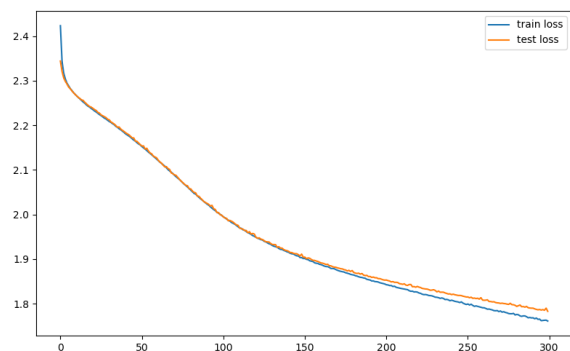
(a) Loss curve



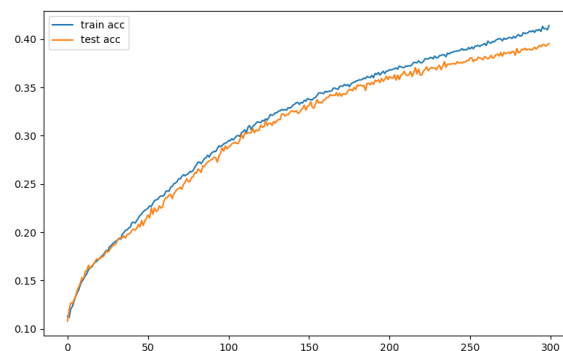
(b) Accuracy curve

Figure 19: Training and test performance curves for VGG-19 using MUON. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

B.3 ResNet-34

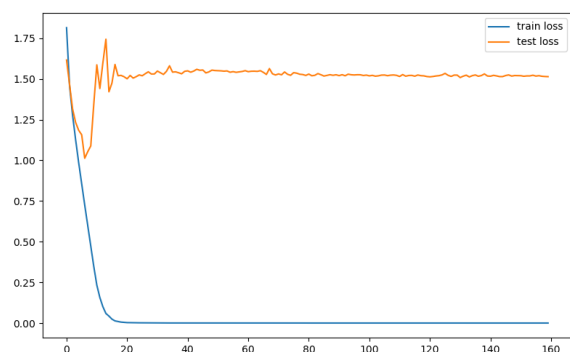


(a) Loss curve

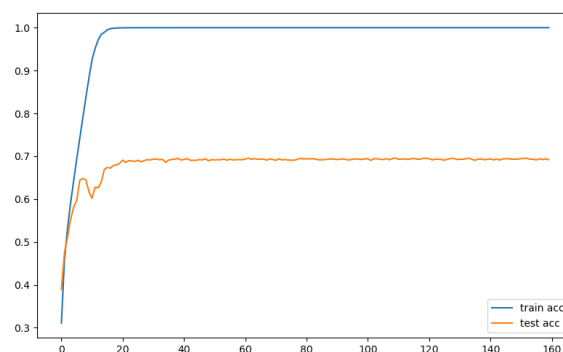


(b) Accuracy curve

Figure 20: Training and test performance curves for ResNet-34 using full-batch GD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

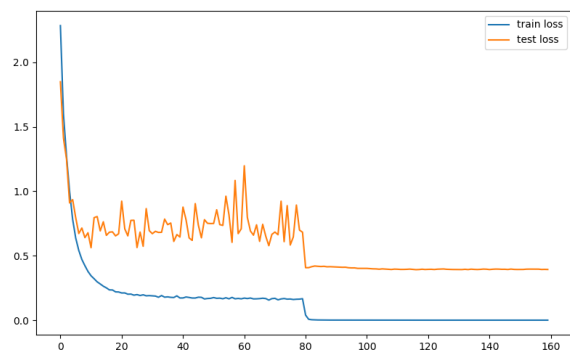


(a) Loss curve

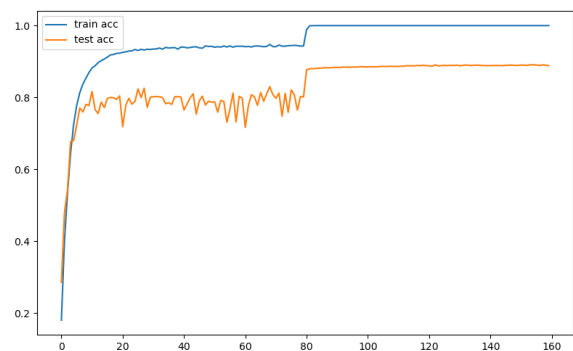


(b) Accuracy curve

Figure 21: Training and test performance curves for ResNet-34 using SGD. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

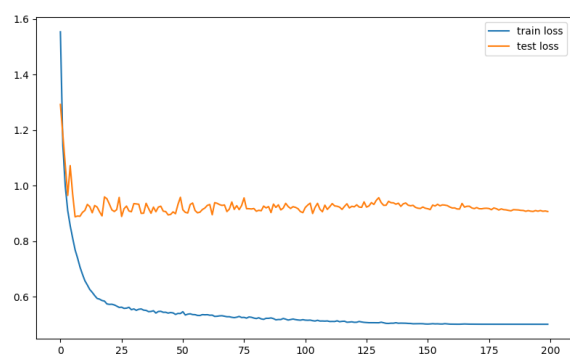


(a) Loss curve

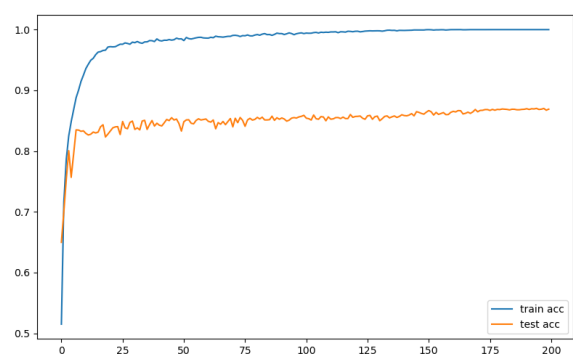


(b) Accuracy curve

Figure 22: Training and test performance curves for ResNet-34 using SGD with momentum. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

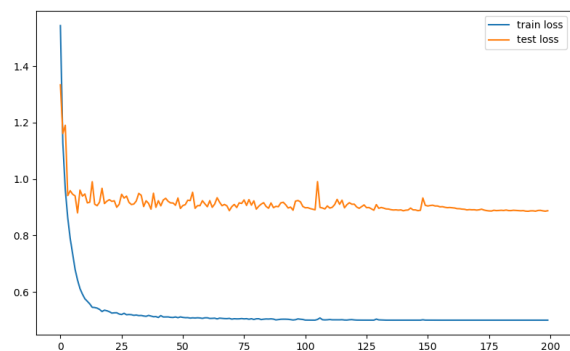


(a) Loss curve

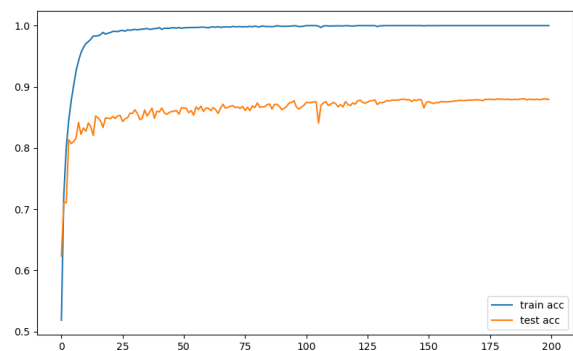


(b) Accuracy curve

Figure 23: Training and test performance curves for ResNet-34 using Adam. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

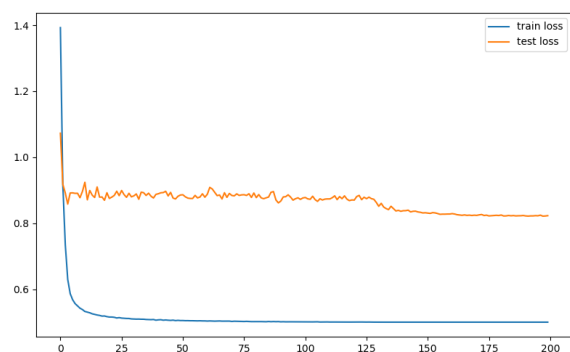


(a) Loss curve

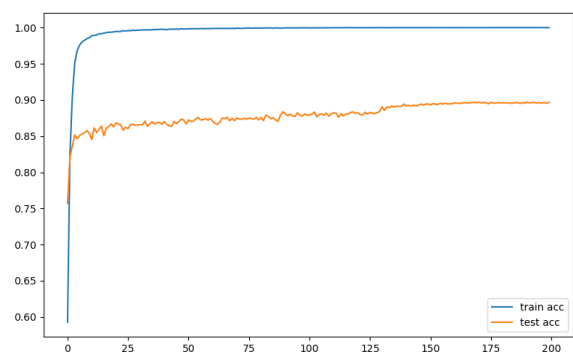


(b) Accuracy curve

Figure 24: Training and test performance curves for ResNet-34 using Adamw. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.



(a) Loss curve

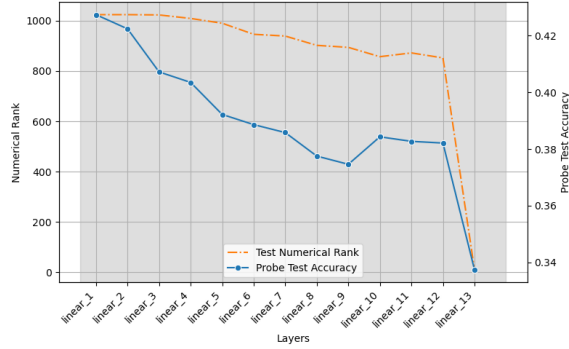


(b) Accuracy curve

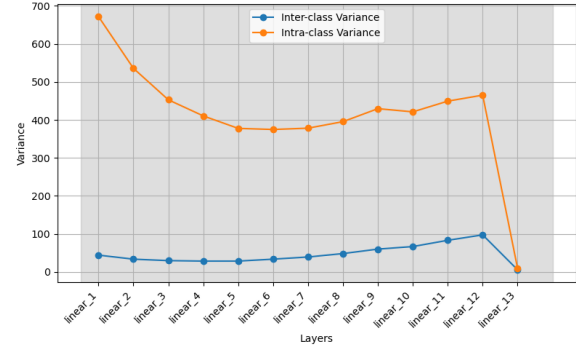
Figure 25: Training and test performance curves for ResNet-34 using MUON. **Left:** Training (blue) and test (orange) loss. **Right:** Training (blue) and test (orange) accuracy.

C Tunnel Analysis

C.1 MLP-12

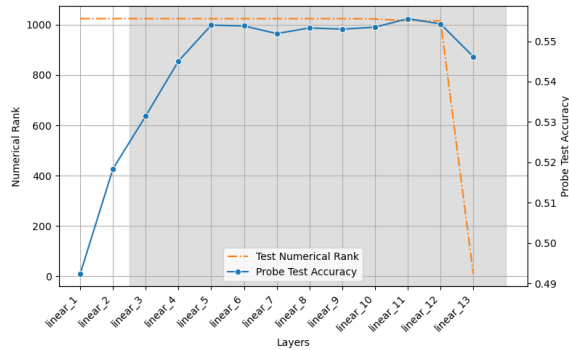


(a) Linear probing accuracy

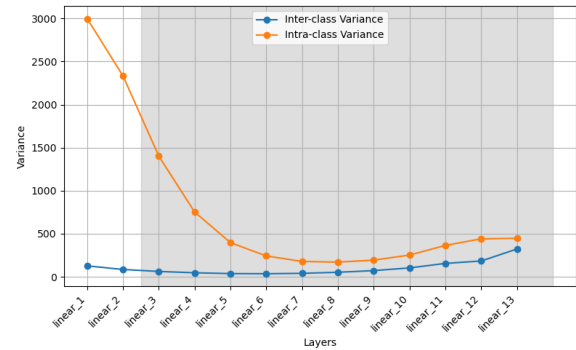


(b) Inter/Intra variance

Figure 26: Analysis of the tunnel effect in the MLP-12 architecture trained with full-batch GD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

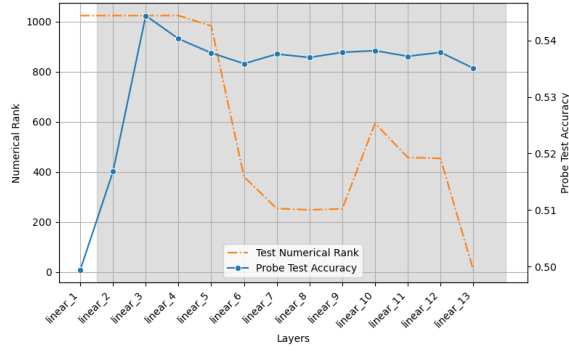


(a) Linear probing accuracy

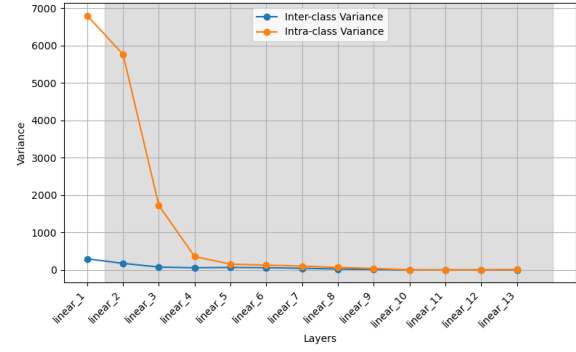


(b) Inter/Intra variance

Figure 27: Analysis of the tunnel effect in the MLP-12 architecture trained with SGD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

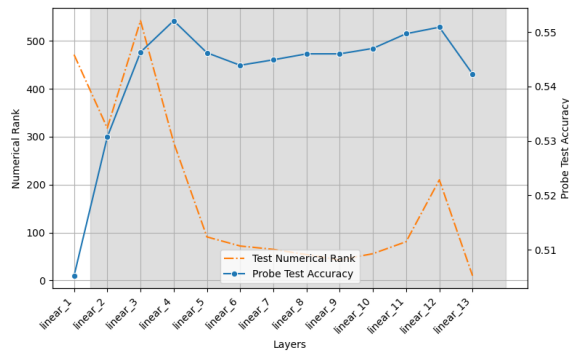


(a) Linear probing accuracy

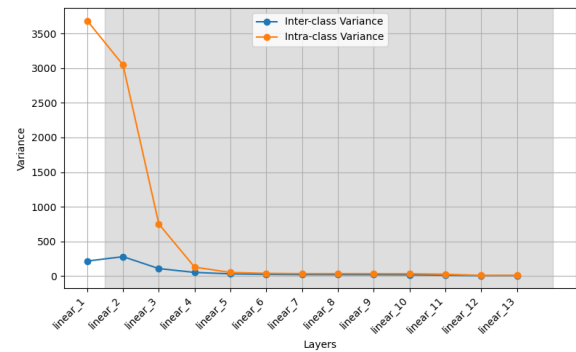


(b) Inter/Intra variance

Figure 28: Analysis of the tunnel effect in the MLP-12 architecture trained with SGD and momentum. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

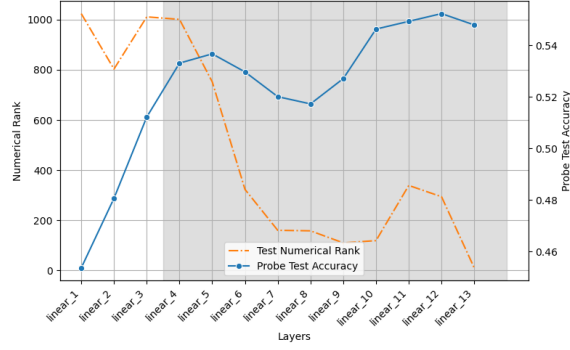


(a) Linear probing accuracy

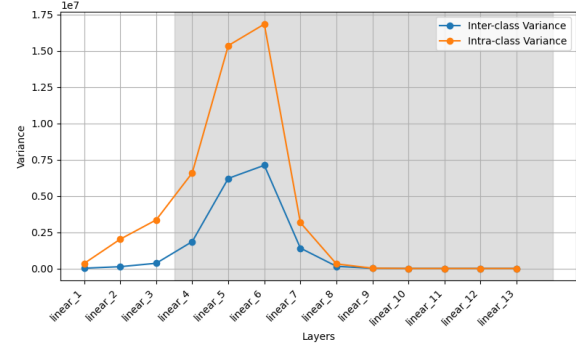


(b) Inter/Intra variance

Figure 29: Analysis of the tunnel effect in the MLP-12 architecture trained with Adam. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

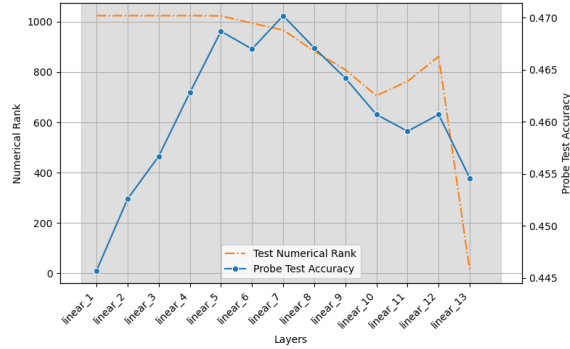


(a) Linear probing accuracy

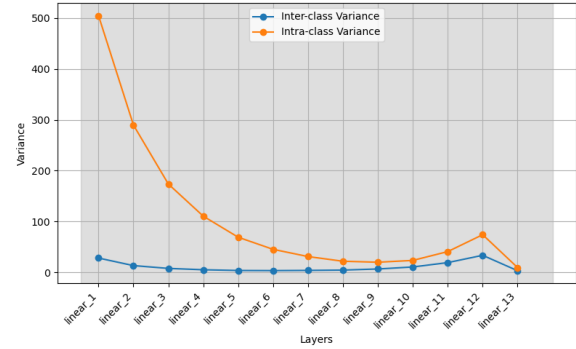


(b) Inter/Intra variance

Figure 30: Analysis of the tunnel effect in the MLP-12 architecture trained with Adamw. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.



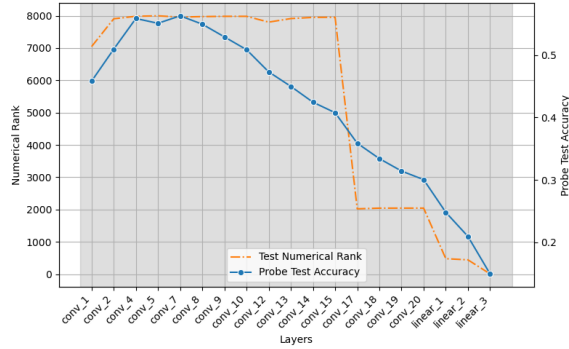
(a) Linear probing accuracy



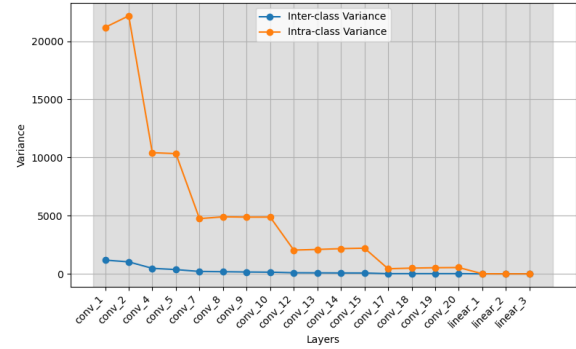
(b) Inter/Intra variance

Figure 31: Analysis of the tunnel effect in the MLP-12 architecture trained with MUON. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

C.2 VGG-19

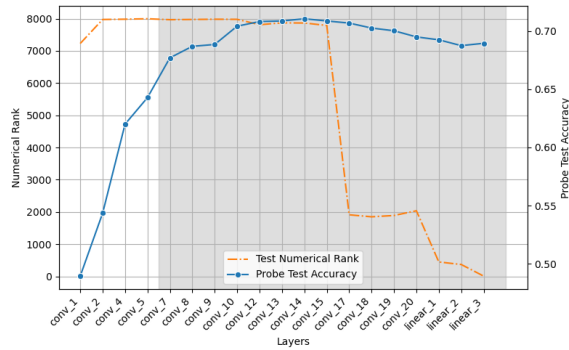


(a) Linear probing accuracy

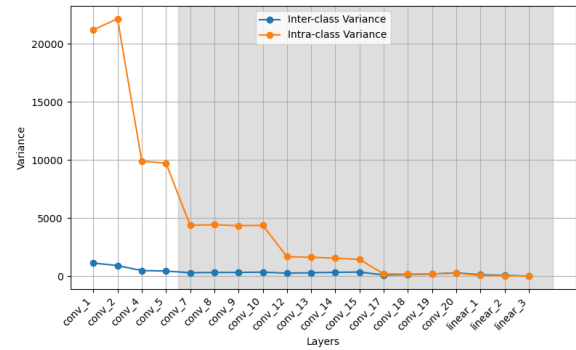


(b) Inter/Intra variance

Figure 32: Analysis of the tunnel effect in the VGG-19 architecture trained with full-batch GD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

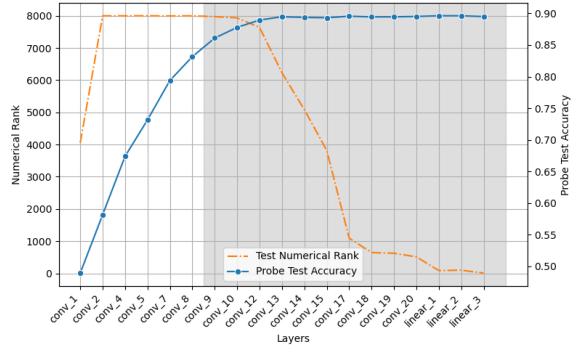


(a) Linear probing accuracy

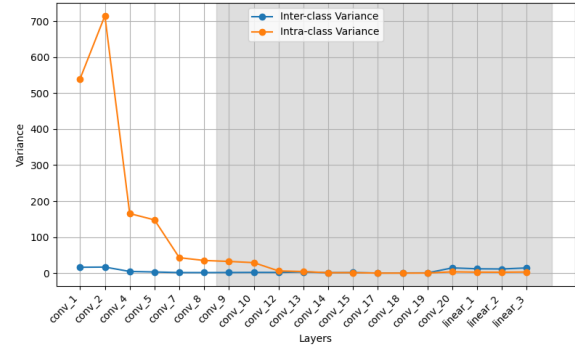


(b) Inter/Intra variance

Figure 33: Analysis of the tunnel effect in the VGG-19 architecture trained with SGD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

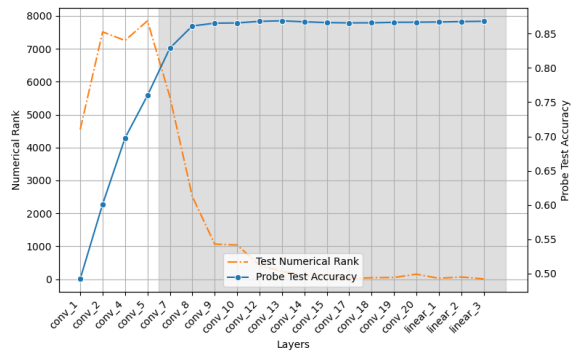


(a) Linear probing accuracy

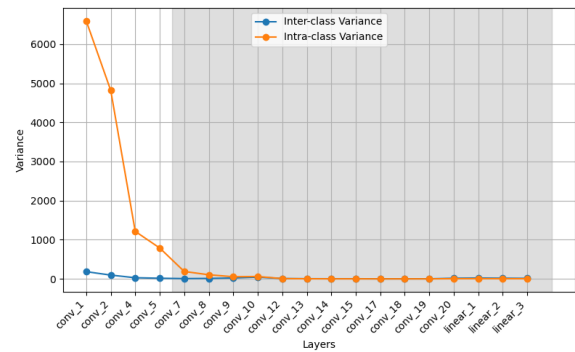


(b) Inter/Intra variance

Figure 34: Analysis of the tunnel effect in the VGG-19 architecture trained with SGD and momentum. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

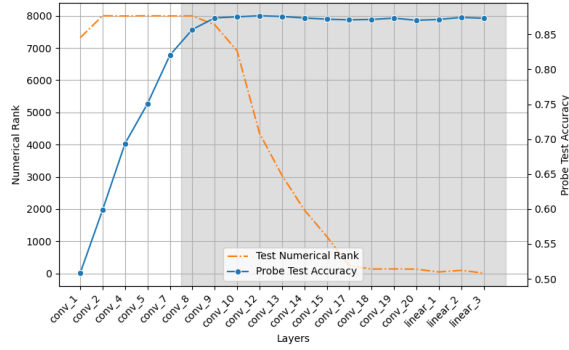


(a) Linear probing accuracy

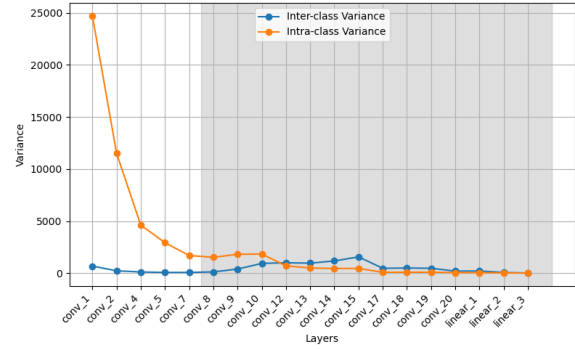


(b) Inter/Intra variance

Figure 35: Analysis of the tunnel effect in the VGG-19 architecture trained with Adam. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

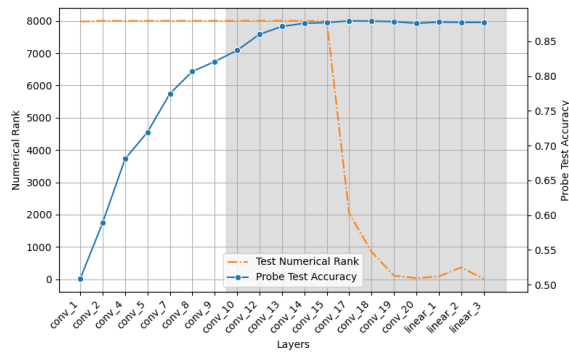


(a) Linear probing accuracy

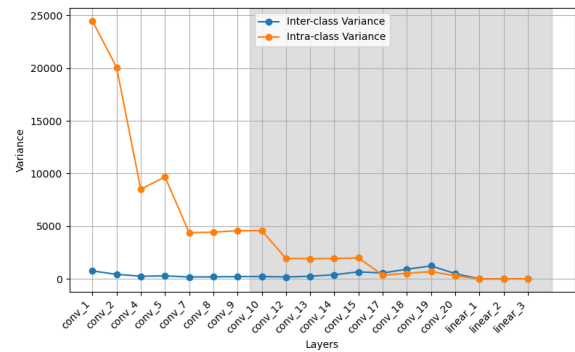


(b) Inter/Intra variance

Figure 36: Analysis of the tunnel effect in the VGG-19 architecture trained with Adamw. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.



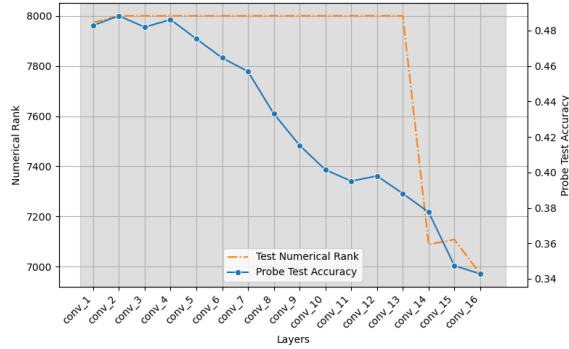
(a) Linear probing accuracy



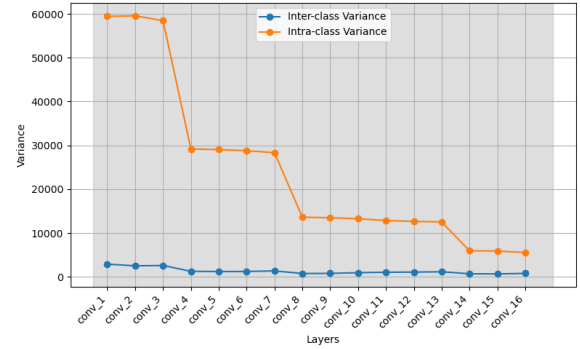
(b) Inter/Intra variance

Figure 37: Analysis of the tunnel effect in the VGG-19 architecture trained with MUON. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

C.3 ResNet-34

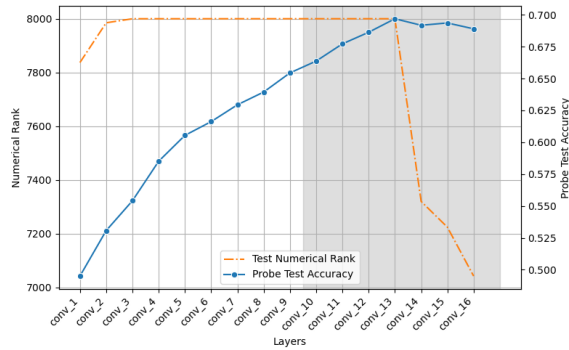


(a) Linear probing accuracy

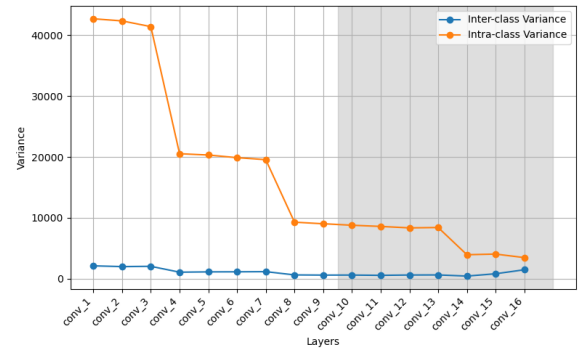


(b) Inter/Intra variance

Figure 38: Analysis of the tunnel effect in the ResNet-34 architecture trained with full-batch GD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

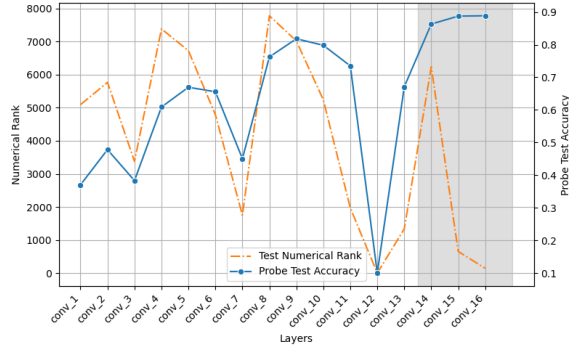


(a) Linear probing accuracy

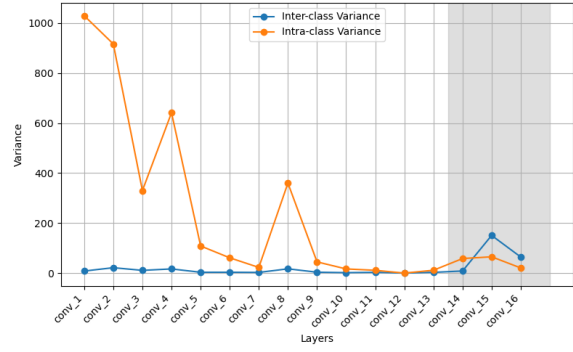


(b) Inter/Intra variance

Figure 39: Analysis of the tunnel effect in the ResNet-34 architecture trained with SGD. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

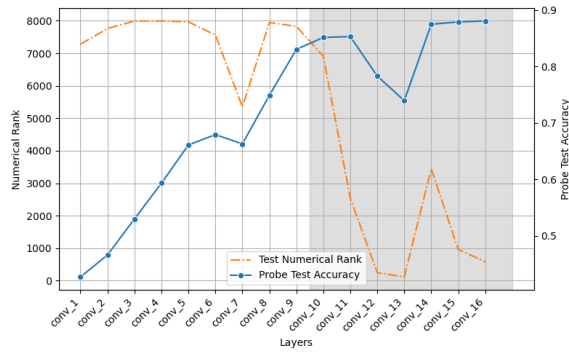


(a) Linear probing accuracy

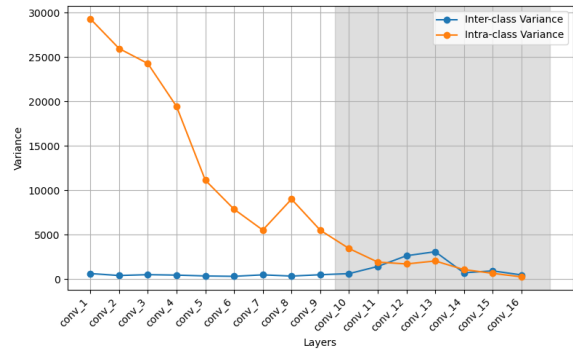


(b) Inter/Intra variance

Figure 40: Analysis of the tunnel effect in the ResNet-34 architecture trained with SGD and momentum. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

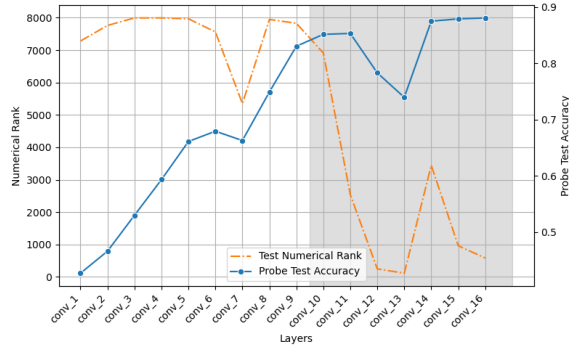


(a) Linear probing accuracy

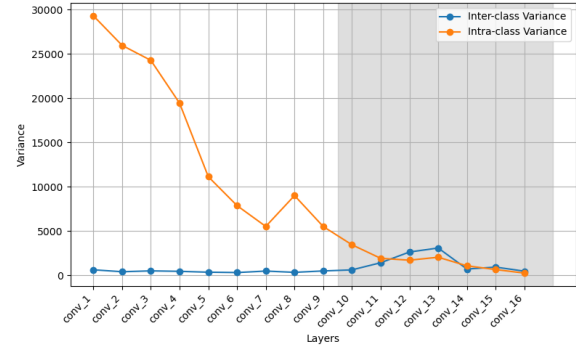


(b) Inter/Intra variance

Figure 41: Analysis of the tunnel effect in the ResNet-34 architecture trained with Adam. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.

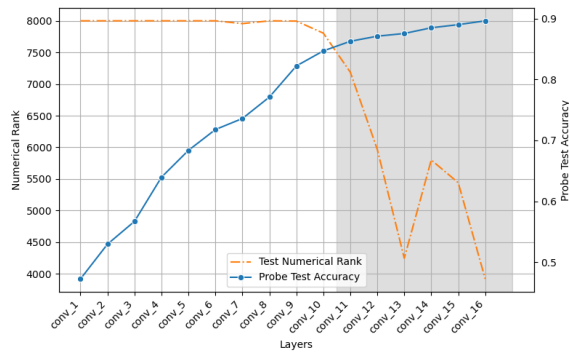


(a) Linear probing accuracy

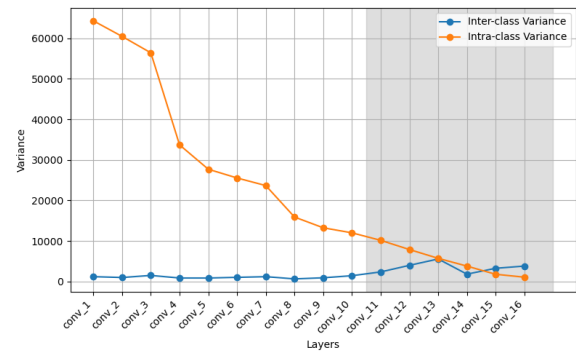


(b) Inter/Intra variance

Figure 42: Analysis of the tunnel effect in the ResNet-34 architecture trained with Adamw. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.



(a) Linear probing accuracy



(b) Inter/Intra variance

Figure 43: Analysis of the tunnel effect in the ResNet-34 architecture trained with MUON. The shaded region represents the tunnel. **Left** Comparative analysis of layer-wise linear probing accuracy (blue) and the numerical rank of representations (orange). **Right** The evolution of inter-class variance (blue) and intra-class variance (orange) across the model hierarchy.